CU SeatFinder

Updated Architecture Decision Records & Microservice Diagram

Group Members

- Chatrin Yoonchalard 6631304721
- Chayaphon Kultanon 6631306021
- Nantaporn Phuaphanbun 6631330021

Architecture Decision Records (ADRs)

ADR-001: Frontend Framework

Context

The occupancy list has to load fast on students' phones over campus Wi-Fi. Options: React SPA (CRA), Vue, Angular, Next.js. An SPA shows a blank screen until its JavaScript loads, and Angular is heavier than a three-person team needs. SEO is not a factor here; the whole app sits behind university SSO (ADR-004), so no page is publicly indexable. Next.js renders the first page on the server while keeping the React component model the team already knows, and its file-based routing and data-fetching conventions remove setup work the team would otherwise write by hand.

Decision

Next.js with server-side rendering. Live occupancy is refreshed by client-side polling, not WebSockets, matching the backend's own polling model (ADR-005).

Status

Accepted

Consequences

- Requires a Node runtime to deploy; static hosting is no longer an option.
- Needs an explicit per-route caching/revalidation policy, or Next.js's default caching will serve stale occupancy numbers.
- The team must learn Next.js's server vs. client rendering model.
- Occupancy is only ever as fresh as the last poll.

ADR-002: Styling Approach

Context

Screens are dense (occupancy badges, filter panels, star ratings) and the team iterates on layout constantly. Options: Ant Design, Bootstrap, TailwindCSS. Ant Design's components resist customization. Bootstrap's styling looks dated. TailwindCSS applies styles as utility classes directly in markup, so layout changes need no separate stylesheet round-trip during iteration, and the app's visual identity is defined by the team rather than inherited from a library's defaults.

Decision

TailwindCSS utility classes, with no component library.

Status

Accepted

Consequences

- Every component (occupancy badges, room cards, rating widgets) must be built from scratch.
- Accessibility (focus states, ARIA patterns) is manual work; no component library supplies it.
- Long utility class lists need agreed conventions to stay readable.
- Visual consistency depends on team discipline rather than an enforced design system.

ADR-003: Backend Language & Framework

Context

The API polls turnstile sources on a schedule and enforces reservation rules. Options: Node/NestJS, Django, Go. Polling a few endpoints every few minutes is not a high-concurrency workload, so throughput is not the deciding factor. Go is chosen for static typing over the reservation logic and single-binary deployment.

Decision

Go, exposing a REST API.

Status

Accepted

Consequences

- Deployment is a single compiled binary, with no language runtime to install on the server.
- Smaller package ecosystem than Node; some utilities will be written in-house.
- No team member has written Go before, so the schedule has to absorb ramp-up on its idioms and concurrency patterns.

ADR-004: Login & Identity

Context

Every student and staff member already holds a university account. Options: build our own email/password signup, phone OTP, or integrate CU's SSO. Our own credential store adds signup friction and one more place passwords can leak; phone OTP carries per-message cost and still doesn't verify that the user is affiliated with the university. CU SSO verifies student/staff status as a side effect of login and keeps passwords out of our system entirely. Admin accounts, however, may belong to library staff outside the SSO directory.

Decision

Authenticate students and staff through CU SSO (OAuth2/OIDC), issuing JWT sessions after login, with a separate email/password path for administrative accounts. During development, run against a local OIDC-compatible mock identity provider so that login-dependent features can be built and tested without network access or real student accounts.

Status

Accepted

Consequences

- Auth has to sit behind an interface from day one so the mock and the real IdP are interchangeable; skipping this makes the swap a refactor.
- Login availability is now tied to the university SSO service's uptime, which the team does not control.
- A second, independent auth path has to be built and secured for admin accounts outside the SSO directory.
- No student password is ever stored, since credential handling is delegated to the IdP.

ADR-005: Turnstile Data Ingestion

Context

No library has granted access to its turnstile/check-in data yet. Options: each library pushes updates to us (webhooks), or our backend pulls counts on a timer. Push requires every library's IT team to build something new before we can launch.

Decision

Poll each library's turnstile data source on a short interval, assuming an accessible export or endpoint can be negotiated.

Status

Proposed. Access has not been arranged, so this cannot be finalized yet.

Consequences

- Access must still be requested from each library's IT/facilities staff: unstarted work, not a formality.
- Needs a scheduler on the backend (Go's time.Ticker or a cron library) to drive polling.
- Missed tap-outs make counts drift upward over time; a reconciliation strategy is required (e.g., daily reset at closing, capping and flagging anomalies).
- Occupancy lags by up to one poll interval.
- If access turns out to be a daily CSV rather than a queryable database, this design has to be revisited.

ADR-006: Database Selection

Context

The system stores three kinds of data: reservations, which need transactional writes so two customers cannot be granted the same room and time slot; reviews, which are relational; and occupancy logs from turnstile polling (ADR-005), which are append-heavy time-series rows read for recent trends.

Options: MongoDB, MySQL, PostgreSQL. MongoDB has no cross-document constraint that prevents a duplicate booking, so the guarantee would have to be enforced in application code - the wrong place for it. MySQL and PostgreSQL both provide transactional writes; PostgreSQL was preferred for richer indexing on timestamp ranges and native JSON columns for the variable tag payloads on reviews.

For hosting, options were Prisma Postgres, Supabase, and Neon (all managed PostgreSQL). Prisma Postgres was chosen for its free tier and minimal setup, since none of these require the team to operate a database server. One consequence of that choice is worth stating: Prisma ORM, the tool usually paired with Prisma Postgres, is Node/TypeScript-only and has no supported Go client, so the Go backend (ADR-003) cannot use it.

Decision

PostgreSQL hosted on Prisma Postgres, queried from Go with a Go-native library (pgx or GORM) rather than Prisma ORM.

Status

Accepted

Consequences

- Row-level locking inside a transaction is what actually enforces the no-double-booking rule for reservations in UC-02.
- No Prisma ORM types or migration CLI on the Go side; migrations need a separate tool (golang-migrate, or Prisma's CLI as a dev-only step).
- Dependent on Prisma Postgres's managed hosting availability and free-tier limits.
- Occupancy logs share the relational database; heavy growth would call for extra indexing or partitioning.

Microservice architecture

Step 1 Identify system operations

System operations are abstractions of external requests derived from the use cases (UC-01 to UC-04) and the actors (Customer, Admin):

- registerUser(), login(): actor authentication via mock OIDC (ADR-004)
- getOccupancy(): UC-01 View Seat Density
- filterLibraries(): UC-03 Filter Empty Libraries
- createReservation() : UC-02 Reserve Meeting Room
- acceptReservation() : UC-02, Admin confirms the booking
- createReview() : UC-04 Review Library
- moderateReview() : UC-04, Admin moderation

Step 2 Identify services

Services are organized around business capabilities rather than technical layers, giving five services:

- User Service: authentication and profile management
- Library Service: library and meeting room master data
- Occupancy Service: turnstile polling and live occupancy calculation (ADR-005)
- Reservation Service: meeting room booking, quota, and QR check-in
- Review Service: ratings, tags, and moderation

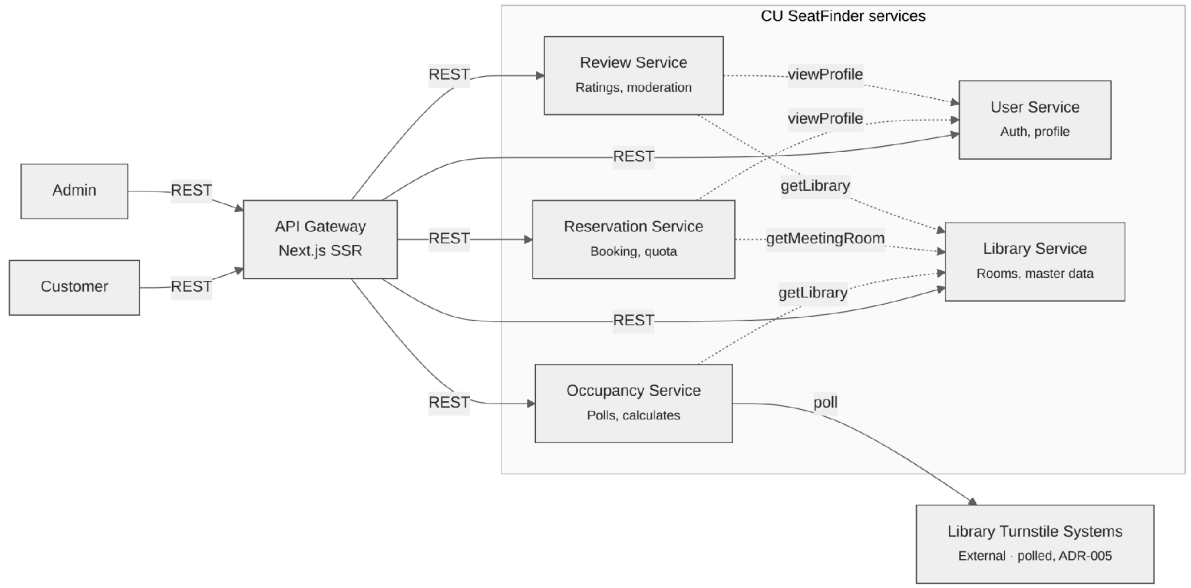
The Occupancy Service is kept separate from the Library Service because the turnstile ingestion pipeline is an unreliable, still-unnegotiated external dependency (ADR-005); isolating it means a polling failure does not take down library browsing.

Step 3 Service APIs and collaborations

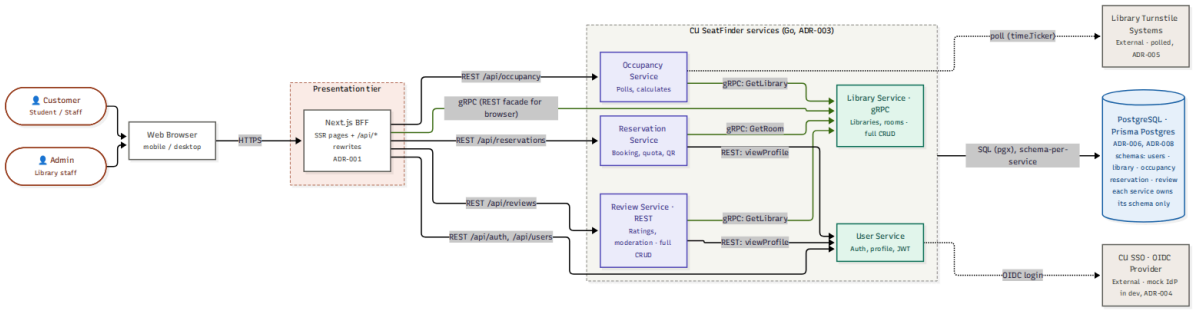
Each row below lists a service, the operations it exposes, and which other services it collaborates with to fulfil each system operation.

Service	Operation	Collaboration
User Service	registerUser() login() logout() editProfile() viewProfile()	Auth interface: mock OIDC identity provider; swapped for CU SSO once credentials are issued (ADR-004)
Library Service	listLibraries() getLibrary() getMeetingRoom() searchLibrary()	source of truth for library, floor, zone, and meeting room master data
Occupancy Service	pollTurnstileData() calculateOccupancy() getOccupancy()	Library Service: getLibrary(): validates the library exists before publishing an occupancy reading
Reservation Service	createReservation() acceptReservation() cancelReservation() checkQuota()	User Service: viewProfile(): checks booking quota / active penalties Library Service: getMeetingRoom(): locks room status inside a transaction (ADR-006 row-level locking)
Review Service	createReview() moderateReview() getReviews() recalculateRating()	User Service: viewProfile(): attaches reviewer identity Library Service: getLibrary(): attaches and recalculates the library's average rating

Architecture diagram



(Previous Architecture)



(Current Proposed Architecture)